

756 Project Progress Report

Aarish, Dhvani, Ankith, Ankush

1. Solution Design

Our system has four components, to be deployed across three GCE VMs.

KV-Store Node: The core server with a two-port design, a Raft TCP port for consensus traffic (leader election, log replication, heartbeats) and a separate gRPC port for client API requests (Get, Set, Delete). We separated these so we can inject faults on the client port without disrupting Raft internals. Uses the HashiCorp Raft library with BoltDB for durable log storage, and an in-memory FSM (key-value map) with per-client sequence numbers for idempotent duplicate detection.

Client CLI: Connects to any node via gRPC; automatically redirects to the leader if it hits a follower.

Chaos Proxy: A TCP proxy that drops connections or adds latency on the gRPC port, simulating network faults without affecting Raft consensus.

Writes flow: Client → Leader (gRPC) → AppendEntries to followers (Raft TCP) → quorum commit → FSM apply → response. Reads go via leader with VerifyLeader() for linearizable consistency. This is a CP system per CAP, the minority side of a partition becomes unavailable rather than serving stale data.

2. Implementation

Stack: Go, gRPC/Protobuf, HashiCorp Raft v1.7.3, BoltDB. Deployment via Dockerfile and a gcloud CLI script that provisions VMs, configures firewall rules, and uploads binaries.

Done so far: The core node is working (leader election, log replication, snapshotting, graceful leadership transfer). The gRPC API, chaos proxy, and dashboard are implemented. We have FSM unit tests and have started writing bash test scripts for Phase 1 (liveness/election) and Phase 2 (network partitions) that work locally against the dashboard API.

Remaining: Deploy to GCP and verify cluster formation across VMs, tune Raft timeouts for real latency, write Phase 3 (latency faults) and Phase 4 (durability) test scripts, adapt existing scripts to use VM IPs instead of localhost, and run the full evaluation.

3. Evaluation Plan

We'll evaluate across four phases on the GCP cluster via bash test scripts and the kv-client CLI

Phase 1 - Liveness/Election: Kill leader via SSH, measure MTTR to new election via Health RPC polling. Verify quorum loss correctly blocks writes.

Phase 2 - Network Partitions: SIGSTOP/SIGCONT to freeze nodes. Verify no split-brain on minority partition, leader step-down on majority partition, and log catch-up after healing.

Phase 3 - Resource/Latency: Chaos proxy with 2s follower delay, 500ms leader delay, 50% packet loss (partial success, all acks durable).

Phase 4 - Durability: Kill all nodes and restart, verify all keys recovered from BoltDB. Kill leader mid-write and verify zero acknowledged-write loss.

Key metrics: MTTR (seconds), write throughput (ops/sec) under faults, and key recovery ratio (target: 100% for acknowledged writes). Results will compare localhost vs. GCP behavior.